

Hyena cho tiếng Việt: Mô hình ngôn ngữ tích chập dài dưới bậc hai, phân tích ablation và khởi tạo bộ lọc từ cấu trúc thông tin của corpus

Nguyễn Cao Trung Kiên Tô Huỳnh Minh Tiến Trần Tú Quang

Trường Đại học Công nghệ Thông tin, ĐHQG-HCM

{250201069, 250201095, 250201084}@grad.uit.edu.vn

Tóm tắt nội dung

Toán tử Hyena (Poli et al., 2023) thay self-attention bằng đệ quy đan xen tích chập dài tham số hoá ngầm và cổng nhân điều khiển bởi dữ liệu, cho chi phí dưới bậc hai theo độ dài chuỗi; mọi thực nghiệm ngôn ngữ của bài báo gốc đều trên tiếng Anh. Chúng tôi cài đặt lại Hyena kèm bộ kiểm thử tính nhân quả, áp dụng cho mô hình ngôn ngữ tiếng Việt với nhánh đối chứng tiếng Anh, và đề xuất khởi tạo phân bố suy giảm của bộ lọc từ đường cong thông tin tương hỗ đo trên corpus. Ở khoảng 7,5 triệu tham số, cùng 50 triệu token và cùng siêu tham số, Hyena có perplexity thấp hơn Transformer 19,5% trên tiếng Việt và 14,7% trên tiếng Anh, nhưng ưu thế này không chuyển thành ưu thế phân loại trên UIT-VSFC. Tiếng Việt mang thông tin tương hỗ cục bộ gấp 1,7 đến 2,3 lần tiếng Anh ở khoảng cách ngắn. Trên tiếng Việt, đặt phân bố suy giảm theo một nguyên tắc, cách đều theo thang log hoặc suy từ corpus, giảm khoảng 0,9 PPL so với một khoảng đặt tùy tiện; trên tiếng Anh hiệu ứng cùng chiều nhưng không qua hiệu chỉnh so sánh bội. Khởi tạo từ corpus không vượt được đường cơ sở thang log một cách có ý nghĩa. Trên nhánh tiếng Anh, dùng α suy từ tiếng Việt không làm perplexity xấu đi có ý nghĩa ($p = 0,28$); chiều hoán đổi ngược lại chưa được kiểm tra.

1 Giới thiệu

Cơ chế self-attention (Vaswani et al., 2017) có chi phí tính toán và bộ nhớ tăng theo bình phương độ dài chuỗi, đặt ra giới hạn cứng cho lượng ngữ cảnh mà mô hình có thể xử lý. Nhiều hướng nghiên cứu tìm cách phá vỡ rào cản này, trong đó dòng mô hình không gian trạng thái (Gu et al., 2022; Mehta et al., 2022; Fu et al., 2023) và tích chập dài nổi lên như những lựa chọn thay thế nghiêm túc.

Hyena (Poli et al., 2023) là một trong những kiến trúc đầu tiên không dùng attention dày đặc mà vẫn đạt chất lượng tương đương Transformer trên mô hình hoá ngôn ngữ ở quy mô lớn. Toán tử này đan

xen tích chập dài tham số hoá ngầm với cổng nhân điều khiển bởi dữ liệu, cho chi phí $O(NL \log_2 L)$ thay vì $O(L^2)$. Dòng nghiên cứu này về sau dẫn tới các kiến trúc chọn lọc trạng thái như Mamba (Gu and Dao, 2023).

Toàn bộ thực nghiệm ngôn ngữ trong bài báo gốc thực hiện trên tiếng Anh, cụ thể là WikiText103 (Merity et al., 2017) và The Pile (Gao et al., 2020). Chưa có bằng chứng nào về hành vi của Hyena trên tiếng Việt, một ngôn ngữ có đặc điểm chữ viết khác hẳn: chữ Quốc ngữ tách rời theo âm tiết, nên một từ ghép nhiều âm tiết bị chẻ thành nhiều token khi token hoá theo khoảng trắng. Các mô hình ngôn ngữ tiếng Việt hiện có phần lớn dựa trên kiến trúc Transformer (Nguyen and Nguyen, 2020).

Bài báo cáo này đóng góp ba điểm.

Thứ nhất, chúng tôi cài đặt lại toán tử Hyena bám sát đặc tả gốc và kiểm chứng tính nhân quả của cài đặt bằng bộ kiểm thử tự động, trong đó có phép kiểm dựa trên gradient kèm một phép kiểm ngược cố ý cài lỗi. Thứ hai, chúng tôi đo đường cong suy giảm thông tin tương hỗ trên corpus tiếng Việt và tiếng Anh, dùng nó để khởi tạo phân bố suy giảm của bộ lọc Hyena thay cho việc đặt tham số này bằng trực giác. Thứ ba, chúng tôi báo cáo đầy đủ các kết quả không ủng hộ giả thuyết ban đầu của chính mình: khởi tạo từ corpus không vượt được một đường cơ sở không dùng thông tin ngôn ngữ, và mức độ đặc thù ngôn ngữ của bộ lọc phụ thuộc vào quy ước ánh xạ nhiều hơn chúng tôi dự kiến.

2 Nền tảng: toán tử Hyena

2.1 Định nghĩa

Gọi $u \in \mathbb{R}^{L \times D}$ là chuỗi đầu vào. Hyena bậc N trước hết tính $N + 1$ phép chiếu tuyến tính của đầu vào, tiếp theo là một tích chập ngắn theo kênh, thu được $(x^1, \dots, x^N, v)$. Toán tử được định nghĩa

bằng đệ quy (Định nghĩa 3.1 trong Poli et al., 2023):

$$\begin{aligned} z_t^1 &= v_t \\ z_t^{n+1} &= x_t^n \cdot (h^n * z_t^n), \quad n = 1 \dots N \\ y_t &= z_t^{N+1} \end{aligned} \quad (1)$$

trong đó $*$ là tích chập nhân quả dài và h^n là các bộ lọc học được. Mỗi tích chập được tính trong miền Fourier, cho độ phức tạp tổng cộng $O(NL \log_2 L)$.

Bài báo gốc chỉ ra rằng Hyena bậc 1 tương ứng với GSS (Mehta et al., 2022) và bậc 2 tương ứng với H3 (Fu et al., 2023), nên bậc N là một trực thiết kế có ý nghĩa chứ không phải tham số tùy tiện.

2.2 Bộ lọc tham số hoá ngầm

Điểm mấu chốt khiến bộ lọc dài mà không tồn tham số theo L là cách tham số hoá ngầm:

$$h_t = \text{Window}(t) \cdot (\text{FFN} \circ \text{PositionalEncoding})(t) \quad (2)$$

với $\text{Window}(t) = \exp\{-\alpha t\} + b$, trong đó α thay đổi theo từng kênh nhằm buộc các bộ lọc có độ dài khác nhau, còn b là độ lệch học được để bộ lọc không bị ép về 0 sau độ dài do α quy định (bài báo gốc cũng thêm số hạng này). Mạng FFN dùng kích hoạt tuần hoàn tần số cao (sine), theo dòng biểu diễn ngầm (Sitzmann et al., 2020; Romero et al., 2022), để khắc phục thiên lệch tần số thấp của mạng nơ-ron. Tích nhân quả được bảo đảm bằng cách đánh giá bộ lọc tại $t = 0 \dots L - 1$ rồi đệm 0 cả tín hiệu lẫn bộ lọc tới độ dài ít nhất $2L - 1$ trước khi biến đổi Fourier.

Phân bố $\{\alpha_i\}$ được mô tả là “thay đổi theo kênh” nhưng bài báo gốc không quy định khoảng giá trị cụ thể, cũng không gắn nó với thống kê của ngôn ngữ đích. Đây chính là khe hở mà đóng góp của chúng tôi nhắm vào.

3 Khởi tạo bộ lọc từ cấu trúc thông tin của corpus

3.1 Động cơ

Với t chuẩn hoá về $[0, 1]$ trên L vị trí, kênh thứ i suy giảm còn $1/e$ tại token thứ khoảng L/α_i . Ta gọi đại lượng này là *độ dài hiệu dụng* của kênh. Câu hỏi đặt ra: nếu đặt tập độ dài hiệu dụng phủ đúng nơi thông tin dự đoán thực sự nằm trong corpus, mô hình có dùng ngân sách kênh hiệu quả hơn không?

3.2 Đo cái gì, và vì sao không dùng bộ tách từ

Phương án hiển nhiên là chạy bộ tách từ tiếng Việt rồi lấy phân bố số âm tiết mỗi từ. Chúng tôi bác bỏ

phương án này vì nó phá hỏng nhánh đối chứng: tiếng Anh không có bộ tách từ tương đương, nên mọi khác biệt quan sát được đều có thể quy cho công cụ đo thay vì cho ngôn ngữ.

Thay vào đó chúng tôi đo trực tiếp thông tin tương hỗ giữa hai token cách nhau d vị trí:

$$I(d) = \sum_{x,y} p_d(x,y) \log \frac{p_d(x,y)}{p(x)p(y)} \quad (3)$$

Cùng một công thức, cùng một quy trình ước lượng, áp cho cả hai ngôn ngữ. Lin and Tegmark (2017) đã khảo sát sự suy giảm của đại lượng này trong văn bản tự nhiên; chúng tôi dùng nó cho một mục đích khác là khởi tạo bộ lọc.

3.3 Xử lý độ chệch của ước lượng

Ước lượng hợp lý cực đại của thông tin tương hỗ luôn chệch dương trên mẫu hữu hạn (Paninski, 2003), và độ chệch tăng theo bình phương cỡ bảng chữ cái; bỏ qua điều này sẽ dẫn tới kết luận rằng ngôn ngữ có corpus nhỏ hơn mang nhiều thông tin tầm xa hơn. Chúng tôi khử độ chệch bằng cách trừ đường nền xáo trộn: xáo ngẫu nhiên corpus phá huỷ mọi cấu trúc thật, nên thông tin tương hỗ đo trên bản xáo chính là độ chệch. Ngoài ra chúng tôi thô hoá bảng chữ cái xuống K ký hiệu thường gặp nhất, làm giảm độ chệch theo K^2 ; thô hoá cho ước lượng thận trọng.

3.4 Hiệu chuẩn công cụ đo

Một phép đo chưa được hiệu chuẩn trên dữ liệu có đáp án biết trước thì không dùng để kết luận được. Chúng tôi kiểm công cụ đo trên bốn chuỗi nhân tạo. Trên chuỗi độc lập hoàn toàn, đáp án đúng là $I = 0$ và bộ đo còn dư 2,82% độ chệch. Trên chuỗi sao chép chu kỳ $k = 8$, nó đạt đỉnh đúng tại $d = 8$ và đỉnh đó nổi 84 lần trên nền. Trên chuỗi Markov, nó cho $I(1) = 2,79$, giảm đơn điệu và tắt ở $d = 64$. Trên chuỗi suy biến toàn 0, nó báo lỗi và từ chối sinh α thay vì trả về một phân bố vô nghĩa.

3.5 Ánh xạ sang phân bố suy giảm

Chúng tôi coi $I(d)$ là khối lượng thông tin nằm tại khoảng cách d , chuẩn hoá thành một phân bố xác suất trên khoảng cách, rồi lấy D phân vị cách đều làm tập độ dài hiệu dụng $\{\ell_i\}$, cuối cùng đặt $\alpha_i = L/\ell_i$. Nơi tập trung nhiều thông tin sẽ được cấp nhiều kênh hơn.

Trong các lần chạy E4, file α của nhánh corpus được sinh từ phép đo BPE với $K = 500$, nhưng mô hình ngôn ngữ chính lại huấn luyện với đơn vị

âm tiết, nên độ dài hiệu dụng đang được chuyển nguyên từ token BPE sang vị trí token âm tiết. Tỷ lệ ký tự trên mỗi token trong file đo là 3,851 so với 4,118 ở nhánh tiếng Việt và 4,155 so với 5,100 ở nhánh tiếng Anh, tức lệch thang khoảng 6,5% và 18,5%. Chúng tôi xem đây là hạn chế về diễn giải, không quy trực tiếp thành sai số perplexity vì chưa có thí nghiệm cô lập yếu tố này.

3.6 Hai đường cơ sở, và vì sao cần cả hai

Chúng tôi so sánh với hai đường cơ sở. Đường thứ nhất, ký hiệu *uniform*, đặt α cách đều tuyến tính trong một khoảng do chúng tôi chọn. Đường thứ hai, ký hiệu *logspace*, đặt độ dài hiệu dụng cách đều theo thang log từ 1 tới L .

Đường *logspace* tồn tại vì một lý do quan trọng. Phép đo cho thấy 80% tới 90% thông tin nằm trong khoảng cách 34 token, tùy ngôn ngữ và bộ token hoá, trong khi cấu hình *uniform* đặt *toàn bộ* 256 kênh ra xa hơn 64 token (Bảng 2). So sánh đề xuất với một cấu hình lệch xa như vậy là so với hình nộm: thắng cũng không chứng minh được gì. Mọi kết luận khoa học trong báo cáo này đều rút ra từ so sánh với *logspace*.

4 Thiết lập thí nghiệm

4.1 Nguyên tắc so sánh công bằng

Trong so sánh chính giữa Hyena và Transformer, mọi cấu hình chỉ khác nhau ở toán tử trộn token. Embedding, pre-LayerNorm, khối MLP, kết nối residual, weight tying, cách khởi tạo và lịch learning rate giữ nguyên vẹn như nhau. Nếu để nhiều thứ khác nhau cùng lúc thì không thể quy chênh lệch perplexity cho toán tử. Riêng các nhánh ablation thay đổi cấu trúc bên trong toán tử nên không giữ nguyên tổng tham số: bậc $N = 1$ ít hơn cấu hình chuẩn 4,4%, bậc $N = 3$ nhiều hơn 4,4%, bỏ positional embedding ít hơn 1,7% (xem Bảng 4).

Chúng tôi cấp positional embedding học được cho cả hai loại mô hình: attention bắt buộc cần thông tin vị trí còn Hyena thì không, nên cấp cho một bên mà không cấp cho bên kia sẽ tạo ra hai biến thay đổi thay vì một; một nhánh ablation riêng kiểm tra Hyena có thật sự không cần. Huấn luyện tính theo ngân sách token cố định chứ không theo số epoch, vì so theo epoch sẽ thiên vị mô hình chạy nhanh hơn.

4.2 Mô hình và dữ liệu

Mô hình 4 lớp, $d_{\text{model}} = 256$, 8 đầu attention, $d_{\text{ff}} = 1024$, $L = 512$, từ điển 16k. Số tham số đo được:

Hyena 7.553.280 và Transformer 7.383.552, lệch 2,30%.

Dữ liệu lấy từ Wikipedia tiếng Việt và tiếng Anh (bản 20231101). Nhánh tiếng Anh được lấy dư 110.000 tài liệu rồi cắt xuống đúng 38.250.964 token của nhánh tiếng Việt, nên hai nhánh có cùng số token huấn luyện duy nhất. Cả hai sau đó huấn luyện trên cùng 49.995.776 token, tức cùng 6.103 bước và cùng khoảng 1,31 epoch.

Chúng tôi khảo sát hai đơn vị token hoá. Thứ nhất là âm tiết, tức tách theo khoảng trắng, vốn là đơn vị tự nhiên của chữ Quốc ngữ. Thứ hai là BPE (Sennrich et al., 2016) mức byte (Radford et al., 2019), huấn luyện trên chính corpus, không sinh token ngoài từ điển. Phân biệt này chỉ tồn tại ở dạng như vậy với tiếng Việt: với tiếng Anh, tách theo khoảng trắng cho ra đơn vị từ chứ không phải âm tiết, nên nhánh đối chứng phải được diễn giải tương ứng.

4.3 Bằng chứng cài đặt đúng

Một mô hình ngôn ngữ nhân quả cài sai, để rò rỉ thông tin từ tương lai, vẫn huấn luyện trơn tru và cho perplexity đẹp bất thường. Đây là lỗi im lặng nguy hiểm nhất của loại nghiên cứu này.

Chúng tôi khử nó bằng bộ kiểm thử tự động, trong đó phép kiểm cốt lõi dùng gradient: lấy đạo hàm của đầu ra tại vị trí t theo đầu vào tại mọi vị trí $s > t$ và đòi hỏi kết quả bằng 0. Attention đạt 0 tuyệt đối nhờ cơ chế che; Hyena đạt dưới 10^{-15} trên mọi biến thể đã kiểm (lần chạy khi viết báo cáo này cho $3,8 \times 10^{-16}$), tức nhân quả tới sai số máy, vì vòng biến đổi Fourier thuận nghịch để lại nhiễu làm tròn.

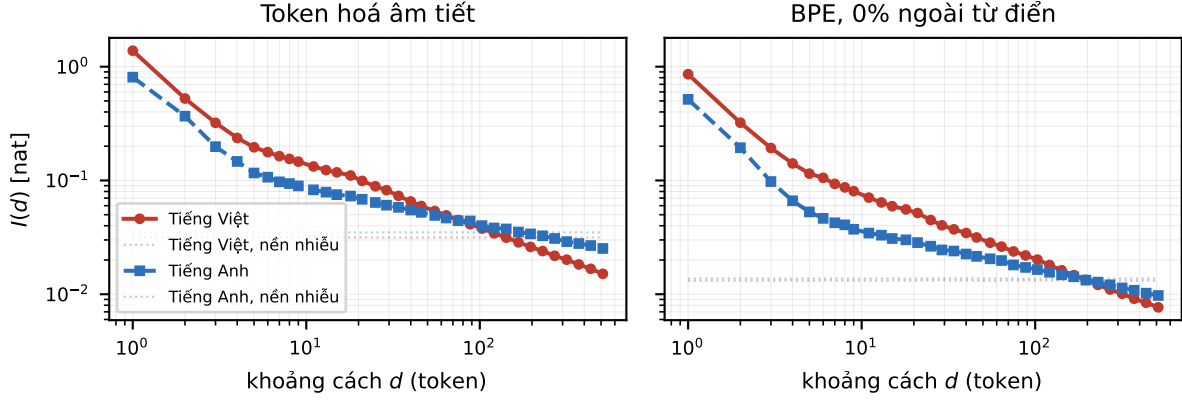
Quan trọng hơn, chúng tôi bổ sung một *phép kiểm ngược*: cố tình cài lỗi nhân quả kinh điển (quên cắt phần đệm phải của tích chập ngắn) và xác nhận tỉ lệ rò rỉ vọt lên khoảng 2. Chênh lệch hơn 15 bậc độ lớn cho thấy bộ kiểm thử thực sự có khả năng phát hiện lỗi, chứ không phải im lặng vì không đủ nhạy.

5 Kết quả

5.1 Cấu trúc thông tin của tiếng Việt so với tiếng Anh

Bảng 1 trình bày tỉ lệ $I_{\text{vi}}(d)/I_{\text{en}}(d)$ đo với hai cách token hoá. Cột thứ nhất dùng token hoá theo âm tiết với từ điển 16k; cột thứ hai dùng BPE mức byte, không sinh token ngoài từ điển, với đúng 10.000.000 token cho mỗi ngôn ngữ.

Ba nhận xét.



Hình 1: Suy giảm thông tin tương hỗ theo khoảng cách. Đường chấm mảnh là nền nhiễu ước lượng; phần đường cong nằm dưới nó không mang thông tin. Với token hoá âm tiết (trái), hai đường cắt nhau quanh $d \approx 88$; khi khử token ngoài từ điển bằng BPE (phải), điểm cắt biến mất và tiếng Việt nằm trên tiếng Anh trong toàn vùng tín hiệu nằm trên nền nhiễu.

d	âm tiết	BPE (0% OOV)
1	1,71	1,66
6	1,66	2,27
13	1,56	1,94
34	1,26	1,55
88	0,94	1,27
167 (dưới nhiễu)		1,03

Bảng 1: Tỷ lệ thông tin tương hỗ giữa tiếng Việt và tiếng Anh theo khoảng cách. Cột âm tiết có nhiều loạn do tỷ lệ token ngoài từ điển chênh lệch (tiếng Anh 10,26% so với tiếng Việt 2,59%); cột BPE khử nhiễu loạn này.

Thứ nhất, với phép đo BPE, tiếng Việt mang nhiều thông tin tương hỗ cục bộ hơn tiếng Anh ở mọi khoảng cách trong vùng đo được, nhưng ưu thế đó tập trung ở khoảng cách ngắn: tỷ lệ đạt 1,66 tới 2,27 ở $d \leq 13$, rồi giảm còn 1,55 ở $d = 34$, 1,27 ở $d = 88$ và 1,03 ở $d = 167$. Với phép đo âm tiết, ưu thế đảo chiều từ $d \approx 88$ (xem nhận xét thứ hai), nên phát biểu này chỉ vững với phép đo BPE. Hình 1 cho thấy trực tiếp điều đó.

Thứ hai, phép đo bằng âm tiết cho thấy hai đường cắt nhau tại $d \approx 88$, nhưng điểm cắt này *biến mất* khi khử token ngoài từ điển. Nó là hiện vật của việc gộp mọi từ hiếm của tiếng Anh vào một ký hiệu duy nhất, làm mất thông tin của tiếng Anh một cách hệ thống. Chúng tôi nêu ra để cảnh báo: nếu chỉ đo bằng âm tiết, chúng tôi đã kết luận sai.

Thứ ba, tín hiệu chìm xuống dưới mức nhiễu ở $d > 121$ với phép đo âm tiết và $d > 167$ với BPE. Mọi phát biểu về khoảng cách xa hơn các mốc này đều không có căn cứ.

Cấu hình	≤ 4	≤ 34	trung vị
uniform	0	0	169,3
logspace	57	145	22,6
corpus, tiếng Việt	142	227	2,95
corpus, tiếng Anh	141	214	2,85

Bảng 2: Số kênh trên tổng 256 có độ dài hiệu dụng dưới ngưỡng cho trước ($L = 512$). Ngưỡng 34 token là nơi chứa 87,5% thông tin với tiếng Việt và 79,9% với tiếng Anh khi đo bằng âm tiết, 88,7% và 83,5% khi đo bằng BPE.

5.2 Phân bố kênh bộ lọc

Bảng 2 so sánh phân bố độ dài hiệu dụng của bốn cấu hình.

5.3 Nhánh đối chứng tiếng Anh và độ bền của nó

Giả thuyết ban đầu của chúng tôi là bộ lọc suy từ corpus mang tính đặc thù tiếng Việt. Theo quy ước ánh xạ dùng khi huấn luyện, bộ lọc suy từ corpus tiếng Việt và tiếng Anh gần nhau: trung vị độ dài hiệu dụng 2,95 so với 2,85 token; trên 256 kênh đã sắp theo phân vị, độ lệch tương đối trung bình $\text{mean}(|\ell_{\text{vi}} - \ell_{\text{en}}|/\ell_{\text{en}})$ là 14,3% (22,3% nếu lấy ℓ_{vi} làm mẫu số, 17,3% với thước đo đối xứng), trong khi bộ lọc tiếng Việt lệch 75,8% so với logspace. Tăng K từ 500 lên 1000 cho 12,9%. Tương quan Pearson giữa các log độ dài không phân biệt được hai trường hợp (0,9974 với cặp Việt và Anh, 0,9377 với cặp Việt và logspace) vì mọi vector phân vị đều đơn điệu, nên chúng tôi không dùng nó làm bằng chứng.

Tuy nhiên kết quả này phụ thuộc một quy ước.

Ngôn ngữ	Mô hình	PPL	KTC 95%
Tiếng Việt	Hyena	51,38 $\pm$ 0,13	[51,05; 51,71]
	Transformer	63,86 $\pm$ 0,29	[63,14; 64,58]
Tiếng Anh	Hyena	55,82 $\pm$ 0,35	[54,96; 56,68]
	Transformer	65,43 $\pm$ 0,18	[64,99; 65,87]

Bảng 3: Perplexity trên tập kiểm tra, 3 seed cho mỗi cấu hình; $\pm$ là độ lệch chuẩn mẫu. Khoảng tin cậy 95% theo phân phối t ($t = 4,303$).

Hàm `alphas_from_mi` coi mỗi điểm trên lưới log của d là một khối lượng xác suất mà không nhân với bề rộng ô quanh điểm đó; lưới log dày ở khoảng cách gần nên phân bố bị kéo về các lag nhỏ. Khi nhân bề rộng ô, trung vị thành 90,5 token với tiếng Việt và 160,5 với tiếng Anh, độ lệch tương đối tăng lên 36,2% (81,1% và 48,9% theo hai thước đo còn lại), còn độ lệch so với `logspace` là 163,3%. Hai bộ lọc vẫn gần nhau hơn so với đường cơ sở, nhưng không còn gần như trùng nhau, và bộ lọc tiếng Việt ngắn hơn, cùng chiều với việc tiếng Việt có nhiều thông tin tương hỗ cục bộ hơn. Vì vậy phân tích hình dạng bộ lọc *không* đủ để bác bỏ tính đặc thù ngôn ngữ.

Phép thử ở mức huấn luyện là hoán đổi α : mô hình tiếng Anh dùng α suy từ tiếng Việt (cùng corpus, cùng cấu hình, 3 seed) đạt 54,77 PPL so với 54,91 khi dùng α của chính nó ($\Delta = -0,15$; KTC 95% $[-0,48; 0,18]$; Welch $p = 0,28$). Vì perplexity ít nhạy với α (mục 5.7), kết luận đúng được chỉ là trên tiếng Anh, α suy từ tiếng Việt dùng thay được α của chính nó, không phải hai bộ lọc giống nhau; chiều ngược lại chưa kiểm được trên đúng corpus. Đối chứng chạy lại cùng phiên cho perplexity trùng khớp tới từng chữ số với lần chạy gốc ở môi trường khác, nên chênh lệch không đến từ môi trường. Các số được tái sinh bởi `python -m hyena_study.analyze`.

5.4 Perplexity trên mô hình ngôn ngữ

Bảng 3 trình bày kết quả chính. Mỗi cấu hình chạy 3 seed với cùng 50 triệu token huấn luyện.

Hyena có perplexity thấp hơn Transformer 19,5% trên tiếng Việt và 14,7% trên tiếng Anh (Welch, $p < 10^{-4}$ ở cả hai).

Bài báo gốc chỉ khẳng định Hyena *tương đương* Transformer ở quy mô 125 đến 355 triệu tham số; kết quả này thuộc chế độ khác nên không mâu thuẫn, nhưng cần đọc thận trọng vì hai lý do. Cả hai kiến trúc dùng chung learning rate và lịch huấn luyện; bài báo gốc cũng làm vậy nhưng ghi nhận Hyena hưởng lợi từ learning rate thấp hơn đôi chút.

Cấu hình	PPL	Δ	KTC 95%	p_{Holm}
Chuẩn (bậc 2, đủ)	51,38		[51,05; 51,71]	
Bỏ cửa sổ suy giảm	53,30	+1,92	[52,24; 54,35]	0,006
Bậc $N = 1$	53,11	+1,73	[51,72; 54,49]	0,024
Bỏ kích hoạt sine	51,80	+0,42	[51,49; 52,10]	0,047
Bậc $N = 3$	51,63	+0,25	[49,51; 53,76]	0,345
Bỏ positional emb	50,79	-0,59	[50,14; 51,45]	0,024

Bảng 4: Ablation trên tiếng Việt, 2 seed mỗi nhánh (cấu hình chuẩn: 3 seed, khoảng α do chúng tôi chọn). KTC 95% theo phân phối t . p_{Holm} : Welch hai mẫu so với cấu hình chuẩn, hiệu chỉnh Holm cho 5 phép so sánh. Với 2 seed, bậc tự do Welch chỉ 1,4 đến 3,0 nên mọi kết luận ở bảng này là sơ bộ.

Và Transformer chạy nhanh hơn 1,51 lần mỗi token (mục 5.8), nên ở cùng thời gian thực nó xử lý được nhiều token hơn. Phát biểu đúng là “Hyena vượt Transformer ở cùng số token, cùng siêu tham số và cùng quy mô này”. Khoảng cách trên tiếng Việt lớn hơn trên tiếng Anh, nhưng hai corpus khác nhau và tỉ lệ token ngoài từ điển chênh lệch (3,1% so với 10,5%), nên chúng tôi không diễn giải chênh lệch này.

5.5 Ảnh hưởng của đơn vị token hoá

Ở cùng lượng ký tự, tiếng Việt sinh nhiều token hơn: trên chính corpus huấn luyện, 4,118 ký tự trên mỗi token so với 5,100 của tiếng Anh, tỉ lệ 1,239.

Chỉ xét tiếng Việt, với token hoá âm tiết Hyena đạt 51,38 so với 63,86 của Transformer, tỉ lệ 1,243; với BPE là 80,02 so với 96,22, tỉ lệ 1,202 (3 seed mỗi ô). Ở đây có một cạm bẫy diễn giải: perplexity *không so sánh được giữa hai bộ token hoá khác nhau* vì từ điển khác nhau cho entropy trên mỗi token khác nhau, nên 51,38 với âm tiết và 80,02 với BPE *không* có nghĩa là âm tiết tốt hơn. Đại lượng so sánh được là chênh lệch log-loss giữa hai mô hình quy về mỗi ký tự: Hyena hơn Transformer 0,053 nat/ký tự với âm tiết và 0,048 nat/ký tự với BPE, cùng chiều và gần nhau. Chúng tôi không ước lượng khoảng tin cậy cho hiệu của hai con số nên không kết luận đơn vị token hoá làm thay đổi lợi thế của Hyena. Cũng lưu ý rằng ở $L = 512$ cố định, hai mô hình nhìn cùng số token, nên tính dưới bậc hai của Hyena không phải là cơ chế giải thích ở đây.

5.6 Ablation các thành phần của Hyena

Từ mục này, mọi so sánh perplexity dùng kiểm định Welch hai mẫu, hiệu chỉnh Holm trong từng họ so sánh ở mức 0,05; khoảng tin cậy trong bảng chỉ để mô tả.

Ngôn ngữ	Khởi tạo	PPL	KTC 95%
Tiếng Việt	uniform	51,38	[51,05; 51,71]
	logspace	50,48	[50,33; 50,62]
	corpus	50,19	[49,93; 50,45]
Tiếng Anh	uniform	55,82	[54,96; 56,68]
	logspace	54,97	[54,56; 55,37]
	corpus	54,91	[54,53; 55,29]

Bảng 5: Ảnh hưởng của cách khởi tạo phân bố suy giảm, 3 seed cho mỗi ô.

Bỏ cửa sổ suy giảm gây thiệt hại lớn nhất (+1,92 PPL), xác nhận vai trò của số hạng Window(t) trong công thức (2). Hạ bậc xuống $N = 1$ cũng gây thiệt hại (+1,73), phù hợp với việc H3, tức Hyena bậc 2, được đề xuất chính vì GSS, tức bậc 1, yếu ở associative recall; nhưng bậc 1 cũng mất 331.776 tham số (4,4%), nên thiệt hại trộn lẫn hai nguyên nhân mà thiết kế hiện tại không tách được. Bỏ kích hoạt sine gây thiệt hại nhỏ (+0,42) và chỉ vừa qua ngưỡng. Bỏ positional embedding lại cho perplexity *thấp hơn* 0,59, phù hợp với lập luận rằng bộ lọc tích chập đã mang thông tin vị trí. Bậc $N = 3$ không khác biệt. Vì bậc tự do Welch rất nhỏ, chúng tôi coi các kết luận này là sơ bộ.

5.7 Khởi tạo bộ lọc từ corpus

Bảng 5 được kiểm bằng Welch với hiệu chỉnh Holm cho sáu phép so sánh cặp.

Trên tiếng Việt, cả logspace lẫn corpus đều vượt uniform, lần lượt 0,90 và 1,19 PPL ($p_{\text{Holm}} = 0,012$ và 0,002). Trên tiếng Anh hiệu ứng cùng chiều và cùng độ lớn, 0,85 và 0,90 PPL, nhưng không qua hiệu chỉnh ($p = 0,034$ và 0,030 trước hiệu chỉnh; $p_{\text{Holm}} = 0,097$). Như vậy khoảng giá trị của α có ảnh hưởng đo được trên tiếng Việt, còn trên tiếng Anh mới là dấu hiệu.

Corpus hơn logspace 0,285 PPL trên tiếng Việt ($p = 0,024$ trước hiệu chỉnh, $p_{\text{Holm}} = 0,097$) và 0,05 PPL trên tiếng Anh ($p = 0,70$). Chúng tôi không có căn cứ kết luận rằng suy phân bố suy giảm từ corpus tốt hơn cách đều theo thang log. Điều ngược lại cũng chưa được chứng minh: trên tiếng Việt, khoảng tin cậy 95% chưa hiệu chỉnh của hiệu là [0,07; 0,50] PPL.

Kết quả này còn cho thấy một giới hạn của thiết kế. Bộ lọc corpus lệch logspace 75,8% mà perplexity chỉ khác 0,05 đến 0,29, tức perplexity ở quy mô này ít nhạy với phân bố α trong một vùng hợp lý. Một cơ chế khả dĩ, chưa kiểm chứng, là độ lệch b của cửa sổ và mạng FFN sinh bộ lọc bù lại một phần khởi tạo trong khi huấn luyện. Vì vậy E4

L	tiền		tiền và lùi	
	sdpa	ngây thơ	sdpa	ngây thơ
256	0,16	0,55	0,23	0,47
512	0,22	0,92	0,27	0,82
1024	0,28	1,56	0,33	1,60
2048	0,37	3,77	0,56	3,13
4096	0,69	8,30	1,02	6,47
8192	1,19	OOM	1,88	OOM

Bảng 6: Mức tăng tốc của Hyena so với hai đường cơ sở attention, tính bằng thương số thời gian chạy (số lớn hơn 1 nghĩa là Hyena nhanh hơn). Cột sdpa là attention tối ưu của PyTorch, cột “ngây thơ” là cài đặt materialise ma trận $L \times L$. Batch 8, $d_{\text{model}} = 256$, fp16, T4; mỗi điểm đo 20 lượt sau 5 lượt khởi động. Cài đặt ngây thơ hết bộ nhớ ở $L = 8192$.

không đủ độ nhạy để trả lời câu hỏi bộ lọc có đặc thù ngôn ngữ hay không, và việc phần chênh lệch nhỏ xuất hiện trên cả hai ngôn ngữ không phải là bằng chứng cho điều đó.

5.8 Hiệu năng theo độ dài chuỗi

Ở $L = 512$, trên các lần chạy tiếng Việt của Bảng 3 (Tesla T4), Transformer đạt 138,5 nghìn token mỗi giây còn Hyena 92,0 nghìn, tức Transformer *nhanh hơn* 1,51 lần (nhánh tiếng Anh: 1,43 lần). Điều này không mâu thuẫn với bài báo gốc, vốn chỉ công bố tăng tốc ở độ dài lớn, nơi chi phí bậc hai của attention lấn át chi phí cố định của biến đổi Fourier.

Khi đo hiệu năng, việc chọn đường cơ sở attention nào cũng quyết định kết luận. Bài báo gốc so với FlashAttention (Dao et al., 2022) và tự ghi rõ rằng cài đặt này đã nhanh hơn attention thông thường trong PyTorch từ 2 tới 4 lần. Vì vậy chúng tôi báo cáo cả hai đường cơ sở: một cài đặt materialise ma trận $L \times L$ và hàm scaled_dot_product_attention (sdpa) của PyTorch. Chỉ so với đường cơ sở thứ nhất sẽ thổi phồng mức tăng tốc nhiều lần.

Bảng 6 cho thấy điểm hoà vốn phụ thuộc mạnh vào đường cơ sở. So với cài đặt ngây thơ, Hyena nhanh hơn từ $L = 1024$. So với sdpa, Hyena ngang bằng ở $L = 4096$ với lượt tiến và lùi (1,02 lần, trong phạm vi nhiễu của một lần đo) và nhanh hơn ở $L = 8192$: 1,19 lần với lượt tiến, 1,88 lần với lượt tiến và lùi. Bài báo gốc báo cáo điểm giao giữa Hyena và FlashAttention nằm giữa 4096 và 8196, cùng dải với điểm giao chúng tôi quan sát. Tuy vậy điều kiện đo khác nhau (đường cơ sở, batch 8 so với 64, phần cứng, và torch.fft thay cho kernel CUDA hợp nhất), nên chúng tôi không so trực tiếp 1,88 với mức 2 lần của bài báo gốc.

Về bộ nhớ ở lượt tiến, cài đặt ngay thơ tiêu thụ 30,6, 83,1, 283,6, 1074,5 và 4201,6 MB khi L tăng gấp đôi từ 256 tới 4096; tỉ số giữa hai mức liên tiếp tăng từ 2,7 lên 3,9, tiến về 4 của dạng bậc hai, và hết bộ nhớ ở 8192. Hyena tiêu thụ 36,5, 66,1, 117,7, 224,7, 438,7 và 865,8 MB, tỉ số từ 1,8 tới 2,0, đúng dạng tuyến tính.

5.9 Chẩn đoán trí nhớ tầm xa

Perplexity không phát hiện được việc mô hình mất khả năng tầm xa: một mô hình chỉ nhìn vài token vẫn đạt perplexity khá tốt trên văn bản tự nhiên. Vì bộ lọc suy từ corpus có trung vị độ dài hiệu dụng chỉ 2,95 token, chúng tôi bổ sung tác vụ associative recall, vốn là công cụ chẩn đoán của chính bài báo gốc.

Trước hết chúng tôi xác nhận tác vụ giải được: attention hai lớp đạt độ chính xác 1,000 ở độ dài 65 với mức đoán mờ 0,100. Hai lần đo thăm dò không lưu file kết quả cho thấy attention còn 0,196 ở độ dài 129, và ở độ dài 33 cùng 18.720 bước, bỏ lịch warmup làm độ chính xác rơi từ 1,000 xuống 0,187; mọi lần chạy về sau đều dùng warmup. Ở độ dài 65, mọi cấu hình Hyena đều kém hơn attention và dao động rất mạnh giữa các seed: 0,564 với uniform và 0,170 với logspace (2 seed mỗi cấu hình). Độ lệch giữa các seed lớn hơn khoảng cách giữa bất kỳ cặp cấu hình nào, nên chúng tôi *không* xếp hạng được các biến thể bộ lọc trên tác vụ này.

Với cấu hình suy từ corpus, chúng tôi gộp 5 seed từ hai ngân sách huấn luyện (2 seed ở 18.720 bước và 3 seed ở 37.440 bước, cùng độ dài 65 và cùng cấu hình tác vụ); việc gộp này là một lựa chọn phương pháp nên chúng tôi nêu rõ. Phân bố thu được *lượng cực*: ba lần đạt 0,922; 0,981; 0,987 và hai lần sập về 0,092 và 0,141, không có giá trị trung gian. Gấp đôi ngân sách làm các lần thành công tốt lên (0,922 lên 0,98) nhưng không khử được kiểu hổng, và cùng một seed khởi tạo có thể đổi kết cục khi đổi ngân sách (seed 0: 0,092 thành 0,981; seed 1: 0,922 thành 0,141). Đây là mất ổn định tối ưu hoá chứ không phải hội tụ chậm, và Hyena có đủ năng lực biểu diễn ở độ dài 65. Với phân bố lượng cực, trung bình kèm độ lệch chuẩn là sai lệch; chúng tôi báo cáo tỉ lệ thành công 3 trên 5 và độ chính xác khi thành công 0,963.

5.10 Đánh giá ngoại tại

Perplexity chỉ đo khả năng dự đoán token kế tiếp. Để kiểm tra biểu diễn có dùng được cho tác vụ khác, chúng tôi đánh giá hai checkpoint âm tiết, uniform, seed 0 của Bảng 3 trên UIT-VSFC

(Nguyen et al., 2018), gồm 16.175 câu phản hồi của sinh viên với hai tác vụ phân loại cảm xúc (3 lớp) và chủ đề (4 lớp). Hai kiến trúc dùng chung siêu tham số trong mỗi giao thức: *thăm dò tuyến tính* (đóng băng thân, học một lớp tuyến tính trên trạng thái ẩn của token cuối) và *tinh chỉnh toàn phần*; epoch được chọn theo macro-F1 trên validation, mỗi ô 3 seed, đối chứng bằng cùng kiến trúc với trọng số ngẫu nhiên. Vì bộ lọc Hyena trong cài đặt của chúng tôi phụ thuộc độ dài chuỗi, mọi câu được đệm về đúng $L = 512$ như lúc tiền huấn luyện.

Theo Bảng 7, ở giao thức thăm dò, tiền huấn luyện cải thiện cả hai kiến trúc với khoảng tin cậy tách rời, nhưng Transformer vẫn bằng hoặc cao hơn Hyena: 0,7079 so với 0,6428 trên chủ đề (tách rời), 0,6505 so với 0,6329 trên cảm xúc (chồng lấn), dù checkpoint Hyena có perplexity thấp hơn (51,50 so với 63,91). Ưu thế perplexity vì vậy không chuyển thành ưu thế biểu diễn; một phần khoảng cách có sẵn từ khởi tạo, vì Transformer ngẫu nhiên đã đạt 0,6431 trên chủ đề. Khi tinh chỉnh toàn phần, mọi so sánh đều chồng lấn. Đệm về $L = 176$ thay vì 512 làm macro-F1 thăm dò của Hyena đã tiền huấn luyện giảm 0,064 tới 0,112 ở cả sáu lần chạy, còn Transformer không đổi.

6 Thảo luận

Phát hiện đứng vững nhất của phần đề xuất không nằm ở nơi chúng tôi dự kiến. Khoảng giá trị của tham số suy giảm, một chi tiết bài báo gốc để ngỏ, có ảnh hưởng đo được trên tiếng Việt: chuyển từ một khoảng đặt tùy tiện sang một lựa chọn có nguyên tắc giảm khoảng 0,9 PPL. Việc bổ sung đường cơ sở logspace sau khi phát hiện uniform đặt toàn bộ kênh ra ngoài vùng có thông tin là quyết định tới kết luận: nếu chỉ so với uniform, chúng tôi đã quy cả 1,19 PPL cho đóng góp đề xuất, trong khi so với logspace phần còn lại chỉ 0,29 PPL và không qua hiệu chỉnh so sánh bội.

Câu hỏi bộ lọc có đặc thù ngôn ngữ hay không mới được trả lời một phần: hình dạng bộ lọc phụ thuộc quy ước ánh xạ, còn ở mức huấn luyện, trên nhánh tiếng Anh, dùng α suy từ tiếng Việt không làm perplexity xấu đi có ý nghĩa ($p = 0,28$); chiều hoán đổi ngược lại chưa được kiểm tra.

Phần đặc thù tiếng Việt đứng vững nằm ở phép đo: ở khoảng cách ngắn, tiếng Việt mang thông tin tương hỗ cục bộ gấp 1,7 tới 2,3 lần tiếng Anh, và nếu chỉ đo bằng âm tiết, nhiễu loạn token ngoài từ điển đã khiến chúng tôi kết luận sai về tầm xa.

Hiện tượng lượng cực ở associative recall gợi ý

Kiến trúc	Khởi tạo	Thăm dò tuyến tính		Tinh chỉnh toàn phần	
		Cảm xúc	Chủ đề	Cảm xúc	Chủ đề
Hyena	ngẫu nhiên	0,4551 $\pm$ 0,0513	0,2981 $\pm$ 0,0424	0,7363 $\pm$ 0,0422	0,7562 $\pm$ 0,0218
Transformer	ngẫu nhiên	0,5984 $\pm$ 0,0205	0,6431 $\pm$ 0,0175	0,7591 $\pm$ 0,0256	0,7672 $\pm$ 0,0108
Hyena	tiền huấn luyện	0,6329 $\pm$ 0,0464	0,6428 $\pm$ 0,0143	0,7596 $\pm$ 0,0411	0,7715 $\pm$ 0,0243
Transformer	tiền huấn luyện	0,6505 $\pm$ 0,0019	0,7079 $\pm$ 0,0050	0,7751 $\pm$ 0,0229	0,7718 $\pm$ 0,0349

Bảng 7: Macro-F1 trên tập test UIT-VSFC, trung bình 3 seed tinh chỉnh trên cùng một checkpoint; $\pm$ là nửa độ rộng khoảng tin cậy 95%, không gồm biến thiên của tiền huấn luyện. Đoán lớp đồng nhất đạt 0,2229 (cảm xúc) và 0,2099 (chủ đề). Chữ đậm: tách rời khỏi kiến trúc còn lại.

vấn đề của Hyena ở quy mô nhỏ có thể nằm ở độ ổn định tối ưu hoá chứ không ở năng lực biểu diễn: cấu hình corpus thành công 3 trên 5 lần, với độ chính xác tới 0,98.

Hạn chế

Quy mô mô hình của chúng tôi (khoảng 7,5 triệu tham số) nhỏ hơn khoảng 47 lần so với quy mô 355 triệu tham số trong bài báo gốc. Mọi kết luận đều gắn với quy mô này và không được ngoại suy.

Chúng tôi dùng `torch.fft` thuần thay cho CUDA kernel tùy chỉnh của bài báo gốc, nên các con số tốc độ không so trực tiếp được với con số trong đó.

Bản tái hiện của chúng tôi *chưa đạt được* kết quả associative recall của bài báo gốc, vốn báo cáo 97,2% ở độ dài 131.072. Đây là hạn chế của bản tái hiện ở ngân sách hiện có, không phải bằng chứng phản bác bài báo gốc. Số liệu associative recall ở 37.440 bước được chép từ nhật ký chạy chứ không phải từ file kết quả gốc.

Do phân bố lưỡng cực và số seed hạn chế, chúng tôi *không* xếp hạng được các biến thể bộ lọc trên tác vụ associative recall. Phân biệt được tỉ lệ thành công giữa các cấu hình đòi hỏi khoảng 10 seed cho mỗi cấu hình, vượt ngân sách tính toán của đề án.

Corpus được lấy mẫu ở chế độ streaming, trong đó việc xáo trộn chỉ diễn ra trong một bộ đệm hữu hạn chứ không xáo toàn cục, nên mẫu lệch về phía đầu tập dữ liệu. Nặng hơn, phép lấy mẫu này không tái lập được giữa các phiên bản thư viện `datasets`. Vì vậy nhánh tiếng Anh phải được lấy mẫu lại (110.000 tài liệu, `datasets` phiên bản 4.0.0, ghi trong mỗi tệp kết quả) rồi cắt xuống đúng ngân sách token của nhánh tiếng Việt, còn nhánh tiếng Việt giữ mẫu gốc; đổi lại hai ngôn ngữ không còn chung một lần lấy mẫu.

Phép đo thông tin tương hỗ có ba điểm yếu chưa xử lý: phép ánh xạ sang α phụ thuộc quy ước rời rạc hoá (mục 5.3); đường nền nhiễu chỉ ước lượng từ một lần xáo; và bước thô hoá về K ký hiệu thường

gặp nhất có thể phủ một tỉ lệ văn bản khác nhau ở hai ngôn ngữ, điều chúng tôi chưa đo nên tỉ lệ 1,7 đến 2,3 lần có thể chịu ảnh hưởng của nó.

Chiều hoán đổi α trên tiếng Việt và seed thứ ba cho ablation không chạy được trên đúng corpus: lấy mẫu lại (`datasets` 4.0.0) cho cùng 90.000 tài liệu nhưng chỉ 3,67 triệu token huấn luyện, khoảng một phần mười bản gốc.

Đánh giá ngoại tại dùng câu ngắn nên không kiểm tra được ngữ cảnh dài, và chỉ có một seed tiền huấn luyện.

Cuối cùng, so sánh “cùng số ký tự” giữa hai ngôn ngữ không phải là “cùng lượng nội dung”. Muốn khẳng định chuỗi tiếng Việt dài hơn ở cùng nội dung cần văn bản song ngữ song song, thứ chúng tôi không có.

7 Kết luận

Chúng tôi cài đặt lại toán tử Hyena kèm kiểm thử tính nhân quả và áp dụng cho mô hình ngôn ngữ tiếng Việt. Ở khoảng 7,5 triệu tham số, cùng số token và cùng siêu tham số, Hyena có perplexity thấp hơn Transformer 19,5% trên tiếng Việt và 14,7% trên tiếng Anh, nhưng ưu thế này không chuyển thành ưu thế phân loại trên UIT-VSFC. Ở độ dài 8192, Hyena nhanh hơn attention tối ưu của PyTorch 1,88 lần cho lượt tiến và lùi. Ablation sơ bộ với 2 seed cho thấy cửa sổ suy giảm có ảnh hưởng lớn nhất; các nhánh khác cần thêm seed.

Đóng góp đề xuất cho kết quả hai mặt. Khoảng giá trị của tham số suy giảm quan trọng: khoảng 0,9 PPL trên tiếng Việt, cùng chiều trên tiếng Anh. Nhưng suy khoảng đó từ thống kê corpus không mang lại lợi ích đo được so với cách đều theo thang log; trên nhánh tiếng Anh, dùng α suy từ tiếng Việt cũng không làm perplexity xấu đi có ý nghĩa, còn chiều hoán đổi ngược lại chưa được kiểm tra. Chúng tôi báo cáo các kết quả phủ định này cùng giới hạn của bản tái hiện thay vì lược bỏ chúng.

Tài liệu tham khảo

- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Daniel Y. Fu, Tri Dao, Khaled K. Saab, Armin W. Thomas, Atri Rudra, and Christopher Ré. 2023. Hungry Hungry Hippos: Towards language modeling with state space models. In *International Conference on Learning Representations (ICLR)*.
- Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. 2020. The Pile: An 800GB dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*.
- Albert Gu and Tri Dao. 2023. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*.
- Albert Gu, Karan Goel, and Christopher Ré. 2022. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations (ICLR)*.
- Henry W. Lin and Max Tegmark. 2017. Critical behavior in physics and probabilistic formal languages. *Entropy*, 19(7):299.
- Harsh Mehta, Ankit Gupta, Ashok Cutkosky, and Behnam Neyshabur. 2022. Long range language modeling via gated state spaces. *arXiv preprint arXiv:2206.13947*.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer sentinel mixture models. In *International Conference on Learning Representations (ICLR)*.
- Dat Quoc Nguyen and Anh Tuan Nguyen. 2020. PhoBERT: Pre-trained language models for Vietnamese. In *Findings of the Association for Computational Linguistics: EMNLP 2020*.
- Kiet Van Nguyen, Vu Duc Nguyen, Phu X. V. Nguyen, Tham T. H. Truong, and Ngan Luu-Thuy Nguyen. 2018. [UIT-VSFC: Vietnamese students' feedback corpus for sentiment analysis](#). In *2018 10th International Conference on Knowledge and Systems Engineering (KSE)*, pages 19–24.
- Liam Paninski. 2003. Estimation of entropy and mutual information. *Neural Computation*, 15(6):1191–1253.
- Michael Poli, Stefano Massaroli, Eric Nguyen, Daniel Y. Fu, Tri Dao, Stephen Baccus, Yoshua Bengio, Stefano Ermon, and Christopher Ré. 2023. [Hyena hierarchy: Towards larger convolutional language models](#). In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. OpenAI technical report.
- David W. Romero, Anna Kuzina, Erik J. Bekkers, Jakub M. Tomczak, and Mark Hoogendoorn. 2022. CKConv: Continuous kernel convolution for sequential data. In *International Conference on Learning Representations (ICLR)*.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*.
- Vincent Sitzmann, Julien N. P. Martel, Alexander W. Bergman, David B. Lindell, and Gordon Wetzstein. 2020. Implicit neural representations with periodic activation functions. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*.